

Delhi Chief Minister Grievance & Complaint Management Dashboard

Master Blueprint for AI-Led Development

Combined Product Strategy, System Design, Implementation Plan, Testing Plan, and GitHub Execution Guide

0. Document Purpose

This document is written as a **single source of truth** for an AI coding agent, a human engineering team, or a hybrid workflow. It is designed to remove ambiguity and define the system from start to finish.

It combines the strongest parts of two approaches:

- a **governance-first product strategy** focused on CM-level operational intelligence, anti-corruption, and field visibility,
- and a **production-grade engineering architecture** built on MERN + Next.js with fallback-first government workflow design.

The intended outcome is not a toy prototype. The intended outcome is a **working MVP that can be demoed, judged, improved, and later deployed by a government team**.

The platform should be built as if it may eventually support:

- millions of citizens,
 - many departments,
 - multiple complaint channels,
 - field officers with low technical literacy,
 - live CM visits,
 - and strict auditability requirements.
-

1. One-Sentence Product Definition

A mobile-first, audit-heavy, fallback-first grievance intelligence system for Delhi that unifies citizen complaint intake, department routing, officer accountability, CM field-visit awareness, resolution verification, and governance analytics in one platform.

2. Strategic Positioning

Do **not** describe the project as a simple complaint portal.

Describe it as:

- a **Chief Minister governance command center**,
- a **civic accountability platform**,
- a **complaint-to-action operating system**,
- a **field intelligence dashboard for public grievance governance**.

This framing matters because the core value is not only complaint submission. The core value is: 1. showing what is happening, 2. showing who owns it, 3. proving what action was taken, 4. preventing false closure, 5. and giving the CM live ground truth.

3. Product Vision

Vision Statement

Build a system that helps the Chief Minister's Office see, prioritize, and act on public grievances with far greater speed, traceability, and truthfulness than conventional ticketing portals.

Product Philosophy

The system should be optimized for:

- **Ease of use** for citizens and staff,
 - **Operational clarity** for senior government users,
 - **Anti-corruption controls** to reduce false closures,
 - **Fallback resilience** when department APIs are unavailable,
 - **Future extensibility** for more departments, complaint types, and integration sources.
-

4. Core Problem Analysis

4.1 Problem the CM actually faces

The CM does not primarily need more raw complaints. The CM needs answers to questions such as:

- Which areas are repeatedly failing citizens?
- Which departments are overloaded?
- Which officers are acting honestly and which are not?
- Are complaints actually resolved on the ground?
- What is the situation near the CM's current visit location?
- Which cases are urgent enough to bypass the normal queue?

- Where is the bureaucracy creating delay or concealment?

4.2 Structural failure in normal grievance systems

Typical grievance systems fail because they often:

- fragment complaints across departments,
- require too much manual sorting,
- allow false closure,
- do not preserve complete action history,
- do not expose officer workload,
- do not adapt well to field visits,
- and fail when API integrations are missing.

4.3 Why this matters at Delhi scale

Delhi is dense, fast-moving, politically visible, and administratively complex. Any serious civic grievance system must handle:

- high complaint volume,
 - many overlapping jurisdictions,
 - multilingual citizens,
 - mobile-first usage,
 - time-sensitive emergencies,
 - and a wide range of infrastructure and service complaints.
-

5. Primary Differentiators

These are the features that should make the system stand out in the evaluation:

5.1 Field Visit Mode

When the CM is in an area, the system shows nearby unresolved and critical complaints within a geofenced radius.

5.2 Citizen Veto on Closure

A complaint is only fully closed after the citizen confirms that the issue was actually resolved.

5.3 Duplicate Complaint Clustering

Many citizens may complain about the same issue. The system merges them into a single master ticket with multiple subscribers.

5.4 Accountability Graph

Every complaint is mapped to a department, officer, status history, evidence trail, and audit log.

5.5 Fallback-First Government Design

If department APIs fail or do not exist, the system still works through an internal employee dashboard.

5.6 Auto-Segregation Engine

The system automatically classifies and routes complaints using rules plus lightweight AI assistance.

5.7 Critical Alert Engine

Life-threatening or high-risk complaints bypass normal workflow and trigger urgent notifications.

5.8 Officer Resource Ledger

The CM can see officer load, bandwidth, open cases, and overload conditions.

6. High-Level Functional Scope

The MVP must support the following broad capabilities: 1. Citizen intake through web, WhatsApp, or operator-assisted entry. 2. Complaint classification, deduplication, and assignment. 3. Department/officer dashboard for active work. 4. CM dashboard with analytics, map views, field visit mode, and alerts. 5. Resolution workflow with evidence and citizen confirmation. 6. Audit logging and anti-fraud safeguards. 7. External API sync where available. 8. Manual fallback when APIs are unavailable.

7. Roles and Permissions

7.1 Citizen

Can submit complaints, view status, receive alerts, confirm or reject resolution.

7.2 CM / CM Office

Can view city-wide analytics, visit-mode map view, critical cases, department overload, and spot directives.

7.3 Department Head / Nodal Officer

Can monitor department queues, reassign work, check SLA breaches, and review officer performance.

7.4 Field Officer / Employee

Can view assigned complaints, update status, upload evidence, and respond to escalations.

7.5 Super Admin

Can configure departments, complaint types, routing rules, SLA policies, and access control.

7.6 System Service Roles

Worker processes for notifications, duplicate clustering, AI classification, report generation, and external sync.

8. Stack Decision in Detail

The challenge explicitly asks for compatibility with **MERN / Next.js**, so that is the baseline architecture.

8.1 Frontend

Next.js + React

Use Next.js because it supports:

- fast page rendering,
- server-side rendering for dashboards,
- route organization,
- mobile-friendly interfaces,
- and a strong admin UI experience.

Use React for:

- reusable dashboard components,
- forms,
- map widgets,
- status boards,
- charts,
- and role-based UI sections.

8.2 Styling and UI toolkit

Tailwind CSS + ShadCN UI

Use this combination because it produces:

- clean government-style UI,
- fast development,

- consistent spacing and typography,
- accessible components,
- and polished dashboards without heavy UI debt.

8.3 Backend

Node.js + Express

Use Node.js because:

- it fits MERN well,
- it handles event-driven requests efficiently,
- it is easy to integrate with Next.js,
- and it is ideal for API-driven systems with queues and webhooks.

Use Express for:

- routing,
- middleware,
- validation,
- auth,
- complaint processing,
- and integration handlers.

8.4 Database

MongoDB

Use MongoDB because grievance data evolves. Different complaint categories may need different metadata fields. MongoDB provides flexibility for:

- complaint schemas,
- history arrays,
- nested attachments,
- dynamic routing metadata,
- and audit objects.

8.5 Cache

Redis

Use Redis for:

- dashboard caching,
- session support,
- rate limiting,
- duplicate search acceleration,
- and background job coordination.

8.6 Queue system

BullMQ

Use BullMQ for:

- notification jobs,
- report generation,
- AI classification tasks,
- API retries,
- citizen confirmation reminders,
- and duplicate clustering tasks.

8.7 File storage

S3 / MinIO

Use object storage for:

- complaint images,
- videos,
- proof of resolution,
- documents,
- and other media.

8.8 Maps and geospatial UI

Leaflet or Mapbox

Use mapping for:

- complaint pins,
- heatmaps,
- geofencing,
- CM visit mode,
- and district/ward visualization.

8.9 Charts and reporting

Recharts or ECharts

Use charts for:

- complaint counts,
- SLA breaches,
- department comparison,
- officer workload,

- trend lines,
- and hotspot analysis.

8.10 Observability

Prometheus + Grafana + Sentry

Use these for:

- metrics,
- dashboards,
- alerts,
- and exception monitoring.

8.11 Deployment

Docker

Use containers to make the app portable and consistent across developer machines and production environments.

8.12 CI/CD

GitHub Actions

Use this for automated builds, tests, linting, and deployment.

9. Why These Technologies for Each Functionality

This section is for the AI agent or human implementer. It explains how each functionality should be built and why.

9.1 Citizen intake

Recommended stack: Next.js form + Node API + MongoDB + BullMQ + Redis + S3

Why:

- Next.js gives a fast and responsive form.
- Node handles submission and validation.
- MongoDB stores complaint data flexibly.
- BullMQ runs notifications and classification asynchronously.
- Redis supports fast deduplication checks and rate limits.
- S3 stores media evidence safely.

9.2 WhatsApp intake

Recommended stack: webhook integration + Node endpoint + queue worker + notification service

Why:

- WhatsApp is likely to be a high-adoption channel.
- The webhook should create complaints and reply with ticket IDs.
- All heavy work should be queued, not done inline.

9.3 Auto-routing

Recommended stack: rule engine in Node + optional AI classifier worker

Why:

- Government routing is often rule-based and jurisdiction-based.
- AI should assist, not replace, routing.
- The system must work even if AI is unavailable.

9.4 Duplicate clustering

Recommended stack: MongoDB geospatial queries + text similarity + Redis cache

Why:

- Similar complaints near each other should be merged.
- This prevents dashboard clutter and preserves one source of truth.

9.5 CM Field Visit Mode

Recommended stack: Next.js map screen + geolocation + geospatial query + complaint heatmap

Why:

- The CM needs nearby complaints, not just an aggregate count.
- The system should support real-time location-driven context.

9.6 Citizen Veto closure

Recommended stack: complaint lifecycle state machine + notification worker + citizen response endpoint

Why:

- Closure should be conditional.
- This prevents desk-based fake closure.

9.7 Officer resource ledger

Recommended stack: aggregate queries + Redis cache + dashboard widgets

Why:

- The CM needs fast visibility into workload.
- This is a governance signal, not just an admin report.

9.8 Critical alerts

Recommended stack: classification rules + priority queue + push notifications

Why:

- Some issues need bypass logic and immediate escalation.
- Time matters more than normal SLA flow.

9.9 Analytics dashboard

Recommended stack: aggregation pipelines + chart components + cache

Why:

- Senior users need a summarized view.
- Fast loading is critical.

10. System Architecture

10.1 Logical layers

1. Presentation layer
2. Application layer
3. Domain/workflow layer
4. Data layer
5. Integration layer
6. Background processing layer
7. Observability layer

10.2 Architecture overview

```
Citizens / Officers / CM
    ↓
Next.js Frontend
    ↓
```

```
Node.js + Express API
  ↓
MongoDB + Redis + S3
  ↓
BullMQ Workers
  ↓
External Integrations
(MCD311 / Open311 / SMS / WhatsApp / Maps)
```

11. Detailed Module Breakdown

The AI builder should treat the platform as a set of modules.

11.1 Authentication Module

Purpose

Provide secure login and access control.

Responsibilities

- OTP login for citizens,
- password or OTP login for staff,
- role checks,
- session handling,
- refresh token rotation.

Data

- user record,
- role,
- permissions,
- login audit.

Edge cases

- expired OTP,
- invalid token,
- inactive user,
- device/session conflict.

Acceptance criteria

- only allowed users can access allowed routes,
- login attempts are logged,
- session expiry is handled cleanly.

11.2 Complaint Intake Module

Purpose

Receive complaints from all supported channels.

Inputs

- text,
- image,
- voice note,
- location,
- optional category hint,
- optional social media capture.

Workflow

- validate input,
- generate complaint reference,
- store complaint,
- send acknowledgment,
- trigger routing.

UX requirements

- minimal fields,
- mobile-friendly,
- clear progress indicator,
- no unnecessary forms.

Acceptance criteria

- user receives complaint ID within seconds,
- stored complaint is retrievable,
- intake works with and without media.

11.3 Auto-Segregation Module

Purpose

Automatically classify and route incoming complaints.

Inputs

- complaint text,
- category hint,

- location,
- attachment metadata,
- language detection result.

Logic

Use a layered strategy: 1. jurisdiction rule, 2. keyword rule, 3. category dictionary, 4. optional AI text classifier, 5. fallback manual review if confidence is low.

Acceptance criteria

- every complaint ends in a candidate department,
 - low-confidence complaints are flagged for review,
 - routing decisions are stored in the audit trail.
-

11.4 Duplicate Clustering Module

Purpose

Merge complaints about the same issue and area.

Workflow

- compute geospatial proximity,
- compare complaint category,
- compare text similarity,
- compare active unresolved tickets,
- create master ticket if match is found.

Rules

- if a similar open complaint already exists in a nearby radius, attach as a subscriber,
- do not create duplicate dashboards for the same incident.

Acceptance criteria

- duplicate flood does not distort analytics,
 - every citizen still receives updates,
 - master ticket is clearly visible.
-

11.5 Department Assignment Module

Purpose

Assign complaints to the correct department or officer.

Responsibilities

- department mapping,
- officer selection,
- workload balancing,
- escalation logic.

Acceptance criteria

- each ticket has a responsible owner,
 - overloaded officers are avoided when possible,
 - reassignment is logged.
-

11.6 Officer Dashboard Module

Purpose

Let staff see and manage assigned work.

Features

- queue view,
- assigned ticket list,
- status update,
- evidence upload,
- note adding,
- escalation request.

UX requirements

- no clutter,
 - large buttons,
 - clear statuses,
 - color coding,
 - low learning curve.
-

11.7 CM Dashboard Module

Purpose

Give the CM a high-level governance view.

Features

- citywide complaint summary,
- heatmap,

- critical alerts,
- officer resource ledger,
- department comparison,
- field visit mode,
- spot directives.

UX requirements

- mobile friendly,
 - single-screen clarity,
 - minimal taps,
 - fast load times.
-

11.8 Field Visit Mode Module

Purpose

Show real-time complaint context around the CM's current location.

Behavior

- detect current location,
- show complaints within configurable radius,
- sort by severity,
- highlight unresolved and critical cases,
- show nearby officers and departments,
- allow spot directives.

Acceptance criteria

- location-based view works on mobile,
 - radius is adjustable,
 - critical cases are prioritized.
-

11.9 Resolution Verification Module

Purpose

Prevent false closure.

Flow

- officer marks ticket provisionally resolved,
- citizen receives proof and prompt,
- citizen confirms or rejects,
- closure occurs only after confirmation or supervisory decision.

Acceptance criteria

- no direct closure without evidence and state tracking,
 - rejection reopens the case.
-

11.10 Critical Alert Module

Purpose

Handle emergencies and high-risk public issues.

Examples

- open manhole,
- live wire,
- waterlogging under bridge,
- gas leak,
- structural danger.

Behavior

- immediate alert,
- red status display,
- leadership notification,
- special SLA.

Acceptance criteria

- critical events are not buried in normal queues.
-

11.11 Analytics Module

Purpose

Generate governance metrics.

Metrics

- total complaints,
- by district,
- by department,
- by category,
- open vs closed,
- SLA breaches,
- average resolution time,
- false closure rate,

- duplicate cluster count,
- officer performance.

Acceptance criteria

- charts render quickly,
 - metrics are filterable,
 - results are exportable.
-

11.12 Integration Module

Purpose

Connect to external systems such as MCD311/Open311 and department APIs.

Rules

- if external APIs exist, sync with them,
- if not, fallback internally,
- never block the core system on API availability.

Acceptance criteria

- integration failure does not crash the complaint flow,
 - sync status is visible,
 - retries are queued.
-

12. Data Model Specification

The AI builder should create these core collections.

12.1 Complaints collection

Required fields:

- id
- referenceNumber
- sourceChannel
- citizenId
- title
- description
- category
- subCategory
- priority
- status

- location
- ward
- district
- departmentId
- officerId
- clusterId
- attachments
- timeline
- citizenVerification
- createdAt
- updatedAt

12.2 Users collection

Required fields:

- id
- name
- phone
- role
- departmentId
- designation
- ward
- isActive
- lastLogin

12.3 Departments collection

Required fields:

- id
- name
- categoriesHandled
- contactMatrix
- slaConfig
- routingRules
- apiConfig

12.4 AuditLogs collection

Required fields:

- id
- entityType
- entityId
- action
- actorId

- timestamp
- metadata

12.5 VisitLogs collection

Required fields:

- id
- cmId
- location
- startTime
- endTime
- notes
- linkedComplaintIds

12.6 NotificationJobs collection or queue metadata

Required for tracking:

- submission notifications,
- assignment notifications,
- escalation notifications,
- closure verification notifications,
- overdue reminders.

13. State Machine for Complaint Lifecycle

A complaint should move through a clearly defined state machine.

Suggested states

- New
- Classified
- Assigned
- InProgress
- AwaitingEvidence
- ProvisionallyResolved
- AwaitingCitizenVerification
- Closed
- Reopened
- Escalated
- MergedIntoCluster

Rules

- Closed should be restricted.

- `ProvisionallyResolved` is not final.
- `Reopened` should preserve history.
- `MergedIntoCluster` should keep citizen linkage.

Why this matters

A state machine prevents random status changes and helps the AI builder create predictable behavior.

14. Business Rules

The AI builder must enforce these rules consistently.

14.1 Complaint ownership

Every complaint must have exactly one responsible current owner, even if multiple departments are involved.

14.2 Closure rule

A complaint can only be fully closed when:

- evidence is present,
- the citizen has confirmed or supervisory closure has occurred,
- and audit log entries are created.

14.3 Duplicate rule

If an open master ticket exists nearby and matches category, merge the new complaint.

14.4 Emergency rule

Critical complaints bypass the normal queue and appear immediately to leadership.

14.5 Fallback rule

If API sync fails, retain the complaint internally and continue execution.

15. API Specification

The AI builder should create a versioned REST API.

Core endpoints

- `POST /api/auth/login`

- POST /api/auth/verify-otp
- POST /api/complaints
- GET /api/complaints/:id
- GET /api/complaints
- PATCH /api/complaints/:id/assign
- PATCH /api/complaints/:id/status
- POST /api/complaints/:id/evidence
- POST /api/complaints/:id/verify
- GET /api/dashboard/cm
- GET /api/dashboard/officers
- GET /api/dashboard/heatmap
- POST /api/integrations/sync
- POST /api/integrations/webhooks/whatsapp

API standards

- validate inputs strictly,
 - return clear error messages,
 - use pagination,
 - version routes,
 - log all mutation calls.
-

16. Frontend Structure and Component Hierarchy

The AI builder should implement a clear UI structure.

16.1 Main routes

- / landing or citizen entry point
- /file-complaint
- /track/:referenceNumber
- /citizen/dashboard
- /employee/dashboard
- /cm/dashboard
- /cm/visit-mode
- /admin/settings

16.2 Component hierarchy

- layout shell
- navigation bar
- status cards
- complaint form
- location picker
- upload widget
- timeline component

- map panel
- heatmap panel
- officer ledger table
- alert banner
- filter sidebar
- report export controls

16.3 UI design rules

- mobile-first,
- high contrast,
- large touch targets,
- minimal jargon,
- role-specific dashboards,
- bilingual labels where possible,
- no wasted screens.

17. Backend Service Breakdown

The AI builder should separate the backend into modules.

Suggested services or logical modules

- auth service,
- complaint service,
- routing service,
- clustering service,
- notification service,
- analytics service,
- integration service,
- audit service,
- upload service,
- visit-mode service.

If using a monolith

Keep a modular monolith with clean folders.

If using service separation later

These modules can be extracted into services without changing the business model.

18. Folder Structure Recommendation

The AI builder should create a repository structure like this:

```
delhi-cm-grievance-dashboard/  
├─ frontend/  
├─ backend/  
├─ docs/  
├─ database/  
├─ infra/  
├─ diagrams/  
├─ scripts/  
├─ tests/  
├─ .github/  
│   └─ workflows/  
├─ docker/  
└─ README.md
```

Inside backend

```
backend/  
├─ src/  
│   ├── config/  
│   ├── controllers/  
│   ├── middlewares/  
│   ├── models/  
│   ├── routes/  
│   ├── services/  
│   ├── workers/  
│   ├── utils/  
│   └─ app.js
```

Inside frontend

```
frontend/  
├─ app/  
├─ components/  
├─ features/  
├─ hooks/  
├─ lib/  
├─ styles/  
├─ public/  
└─ types/
```

19. Redis Usage Specification

Redis should be used for:

- dashboard cache,
- session TTL,
- rate limiting,
- deduplication lookups,
- job locks,
- temporary OTP state,
- live counts.

Recommended caching patterns

- short TTL on summary metrics,
 - cache invalidation after complaint mutations,
 - cached geospatial summaries for CM dashboard,
 - cached officer load metrics.
-

20. BullMQ Job Specification

The AI builder should implement asynchronous jobs for:

- sending acknowledgment messages,
- sending assignment alerts,
- sending escalation alerts,
- generating reports,
- performing AI routing,
- checking for duplicate clusters,
- syncing with external APIs,
- reminding users for resolution verification.

Job priorities

- critical alerts: highest priority,
 - citizen verification reminders: medium-high,
 - report generation: medium,
 - sync retry jobs: medium-low.
-

21. Geospatial Intelligence Design

This system should be treated as a location-aware governance engine.

Required geospatial features

- complaint pins,
- density heatmaps,
- geofence around CM current location,
- radius-based duplicate detection,
- ward and district overlay,
- nearby complaints during visit mode.

Implementation expectation

- store coordinates in complaint records,
 - index geospatial data,
 - query by distance,
 - render map widgets in CM dashboard.
-

22. AI Routing and Classification Strategy

The AI builder must not overcomplicate the AI layer.

Goal of AI

Improve routing quality and reduce manual burden.

Recommended strategy

1. Start with deterministic rules.
2. Add a lightweight text classifier for ambiguous cases.
3. Use confidence thresholds.
4. Fall back to manual review when uncertain.

What the AI should classify

- complaint category,
- priority,
- probable department,
- urgency level,
- duplicate likelihood,
- language/dialect hints.

What the AI should not do

- make final legal decisions,
 - auto-close complaints,
 - replace accountability logic.
-

23. Anti-Corruption and Fraud Prevention Design

This is a critical differentiator.

Controls to implement

- citizen confirmation before final closure,
- mandatory proof upload for closure,
- geo and timestamp validation where possible,
- audit logs for every status change,
- officer history and performance tracking,
- suspicious pattern detection,
- supervisor escalation when verification fails repeatedly.

Suspicious patterns to flag

- repeated quick closures,
 - repeated reopenings,
 - missing proof files,
 - abnormal officer throughput,
 - repeated complaints from same area closing too quickly,
 - high reject rate after citizen verification.
-

24. Officer Workload Balancing Strategy

The AI builder should include a workload-aware assignment strategy.

Inputs

- current open complaints,
- overdue complaints,
- complaint severity,
- officer ward assignment,
- officer specialization,
- department availability.

Assignment strategy

- assign to the most appropriate officer,
 - avoid overloaded officers when alternatives exist,
 - expose load metrics to the CM dashboard,
 - escalate if workload exceeds threshold.
-

25. Critical Life-Threatening Complaint Handling

Examples

- exposed live wire,
- open manhole,
- structural collapse risk,
- severe waterlogging in a critical area,
- gas leak,
- emergency public hazard.

Required behavior

- immediate high-priority classification,
 - color-coded alerting,
 - direct leadership visibility,
 - short SLA timer,
 - escalation if not acknowledged.
-

26. Citizen Verification Workflow

Flow

1. Officer marks issue provisionally resolved.
2. Citizen receives proof and prompt.
3. Citizen confirms or rejects.
4. If confirmed, complaint closes.
5. If rejected, complaint reopens and escalates.

Requirement

The system should keep all timelines intact and never erase prior state history.

27. MCD311 / Open311 and Government API Integration Strategy

Principle

The system should integrate when APIs exist and gracefully degrade when they do not.

Integration expectations

- support mocked APIs during development,
- use real endpoints if available later,
- map complaint fields to external schemas,
- sync status changes both ways when possible,

- retain an internal record regardless of integration status.

Failure handling

- API timeout should not break complaint intake,
 - retry in background,
 - visible sync status,
 - manual override by employee dashboard.
-

28. Fallback Employee Dashboard Strategy

This is essential.

Why it exists

Government APIs may be unavailable, incomplete, or delayed.

What the fallback dashboard must do

- show assigned complaints,
- show complaint details,
- allow status updates,
- allow evidence upload,
- allow comments and reassignment,
- allow manual closure request,
- support supervisor review.

Principle

The platform must still function as the operational source of truth even when external systems are absent.

29. Reporting and Analytics Strategy

Analytics views

- top complaint categories,
- district hotspots,
- department performance,
- officer ranking,
- SLA compliance,
- duplicate cluster volume,
- critical case timeline,
- closure verification outcomes.

Export formats

- CSV,
 - PDF,
 - screenshot-friendly dashboards,
 - printable weekly reports.
-

30. Security and Privacy Requirements

The AI builder must implement security from day one.

Controls

- HTTPS everywhere,
- hashed passwords,
- role-based access,
- input validation,
- output sanitization,
- rate limiting,
- secure file uploads,
- audit trail,
- secret management.

Privacy principles

- collect only essential data,
 - minimize sensitive fields,
 - clearly explain why data is stored,
 - allow government processing where required,
 - protect citizen communication data.
-

31. Observability Requirements

Must include

- error logging,
- request logging,
- queue metrics,
- API response latency,
- worker failure alerts,
- dashboard performance monitoring.

Why it matters

If the system is intended for government use, failures need to be visible and recoverable.

32. Disaster Recovery and Reliability

Required measures

- daily backups,
- restore testing,
- database replication strategy,
- rollback-friendly deployments,
- worker retry logic,
- graceful degradation.

Goal

The system should fail soft, not fail hard.

33. Performance Targets

The AI builder should optimize for:

- fast complaint submission,
- fast dashboard rendering,
- short response times for key summaries,
- no blocking on background tasks,
- efficient analytics caching.

Suggested practical targets

- intake response within a few seconds,
 - critical dashboard widgets load quickly,
 - assignment and routing should feel near-instant to users.
-

34. Testing Strategy

The AI builder must write tests for:

- complaint submission,
- routing logic,
- duplicate clustering,
- citizen verification,
- role-based access,
- API failure fallback,
- queue jobs,
- analytics queries,

- upload validation,
- audit logging.

Test layers

1. unit tests,
2. integration tests,
3. API tests,
4. UI tests,
5. workflow tests,
6. manual demo scenario tests.

Acceptance testing

Each major function should have a clear success condition.

35. GitHub and Commit Strategy

The AI builder should use GitHub as the project record.

Required GitHub practice

- initialize repo early,
- commit each milestone separately,
- keep commits descriptive,
- keep branches for major features,
- document what changed in each phase.

Suggested commit strategy

- chore: initialize project structure
- feat: add complaint intake module
- feat: add officer dashboard and routing
- feat: add cm analytics and visit mode
- feat: add citizen verification workflow
- feat: add duplicate clustering
- fix: improve fallback integration handling
- test: add workflow tests
- docs: update architecture and usage guide

Branch strategy

- main
- dev
- feature/citizen-intake
- feature/cm-dashboard

- feature/integrations
 - feature/testing
-

36. What the AI Should Ask the User For Before Building

Before starting implementation, the AI agent should ask for only the essentials it cannot infer.

Required user inputs

- GitHub repository access or confirmation of repo creation,
- environment variable values,
- SMS/WhatsApp provider keys,
- map provider key if needed,
- domain name if deployment is desired,
- desired branding preferences,
- whether mock APIs should be used first,
- whether the system should be built as monorepo or split repo.

Optional but useful inputs

- official department list,
- sample complaint categories,
- logo files,
- color palette,
- demo seed data,
- preferred deployment provider.

Important principle

The AI should proceed with mocks where possible and only stop for truly necessary secrets.

37. Recommended Prerequisites to Give the AI First

If the user wants the AI to proceed as independently as possible, provide these first: 1. repository name, 2. preferred stack confirmation, 3. environment variable template, 4. sample complaint categories, 5. list of roles, 6. mock data if available, 7. branding/theme preference, 8. whether to use dummy providers initially.

Minimum viable starting pack

- project name,
- stack choice,
- repo access,
- and permission to build with mock integrations.

Secrets can come later

For example:

- WhatsApp API key,
- SMS API key,
- email SMTP,
- map API key,
- deployment secrets.

The AI should not be blocked on these if mock modes are allowed.

38. Phase-by-Phase Execution Plan for the AI Builder

The AI builder should follow this exact order.

Phase 1: Clarification and setup

- confirm repo structure,
- confirm environment variables,
- confirm data model assumptions,
- confirm channels and roles,
- confirm mock vs real integrations.

Phase 2: Project scaffold

- initialize frontend and backend,
- add linting and formatting,
- set up Docker,
- set up base configs,
- create README and docs skeleton.

Phase 3: Core domain model

- implement complaint, user, department, audit models,
- define complaint states,
- define role permissions,
- define routing rules.

Phase 4: Citizen intake

- build complaint form,
- support uploads,
- support OTP,
- create acknowledgment flow,
- create tracking page.

Phase 5: Routing and assignment

- implement auto-segregation,
- implement workload balancing,
- implement manual override,
- implement assignment history.

Phase 6: Employee dashboard

- build staff queue,
- build complaint details page,
- build evidence upload,
- build status updates,
- build escalation actions.

Phase 7: CM dashboard

- build summary cards,
- build heatmaps,
- build resource ledger,
- build field visit mode,
- build critical alerts.

Phase 8: Verification and anti-fraud

- implement provisional resolution,
- implement citizen veto,
- implement closure criteria,
- implement suspicious activity flags.

Phase 9: Integrations

- add webhook handlers,
- add mocked external API sync,
- add retry jobs,
- add notification channels.

Phase 10: Testing and hardening

- run tests,
- fix edge cases,
- verify error handling,
- ensure mobile responsiveness,
- document all workflows.

Phase 11: Demo preparation

- seed demo data,
- prepare walkthrough scenario,

- ensure GitHub commits are clean,
 - prepare screenshots and diagrams,
 - finalize presentation notes.
-

39. Step-by-Step Build Instructions for the AI

The AI builder should be instructed to do the following in order:

1. Create the repo structure.
 2. Set up Next.js frontend.
 3. Set up Node.js backend.
 4. Add config system and environment template.
 5. Create MongoDB models.
 6. Create complaint intake API.
 7. Create routing logic.
 8. Create dashboard shells.
 9. Add map view.
 10. Add queue workers.
 11. Add notifications.
 12. Add citizen verification.
 13. Add audit log middleware.
 14. Add fallback employee dashboard.
 15. Add analytics.
 16. Add CI/CD.
 17. Add Docker.
 18. Add tests.
 19. Add documentation.
 20. Prepare demo data and final walkthrough.
-

40. Acceptance Criteria by Functionality

Complaint intake

- submits successfully,
- stores complaint,
- returns reference ID,
- supports media,
- logs audit record.

Routing

- assigns to department or officer,
- uses fallback when needed,
- handles low-confidence items.

CM dashboard

- loads summary,
- shows hotspots,
- shows workload,
- supports mobile.

Field Visit Mode

- shows nearby complaints,
- displays critical alerts,
- works on mobile.

Citizen verification

- closure only final after confirmation or approved override,
- reopening works.

Duplicate clustering

- merges near-duplicate complaints,
- preserves original citizens.

Fallback integration

- complaint still processes when external API fails.

Audit logging

- every important action is logged.

41. Testing Scenarios the AI Must Simulate

The AI builder should create test cases for these scenarios:

- citizen files complaint with text only,
- citizen files complaint with image and location,
- complaint is routed to the right department,
- duplicate complaint arrives near existing ticket,
- officer marks resolved with proof,
- citizen rejects closure,
- critical complaint appears,
- external API fails,
- employee dashboard handles fallback,
- CM opens visit mode in a location,
- officer overload is visible,
- analytics updates after new complaint,

- unauthorized user tries to access staff dashboard.
-

42. Demo Script for the Final Presentation

Recommended demo order

1. Show citizen complaint submission.
2. Show auto-routing and ticket ID.
3. Show duplicate clustering.
4. Show officer handling the ticket.
5. Show provisional resolution and citizen veto.
6. Show CM dashboard analytics.
7. Show visit mode and nearby complaint detection.
8. Show critical alert handling.
9. Show fallback workflow when API is unavailable.
10. Show audit trail and accountability graph.

Demo goal

The demo must prove three things:

- the system works,
 - the system is realistic,
 - the system solves a real governance problem.
-

43. How the System Should Be Documented Side by Side

The AI builder should also generate documentation as it builds.

Required documentation files

- README.md
- architecture overview
- data model description
- API reference
- deployment guide
- role and permissions guide
- testing guide
- demo walkthrough
- environment variable guide
- integration guide
- security notes
- future roadmap

Why this matters

Good documentation makes the project easier to judge, easier to extend, and easier to hand over to government stakeholders.

44. Future Roadmap

After MVP, the system can expand into:

- more complaint categories,
 - more departments,
 - richer field officer apps,
 - multilingual voice support,
 - advanced reporting,
 - public transparency dashboards,
 - district-level decision support,
 - machine-assisted trend prediction,
 - and deeper integrations with government systems.
-

45. Final Build Guidance for the AI

The AI must optimize for these priorities in order: 1. correctness, 2. reliability, 3. auditability, 4. usability, 5. performance, 6. extensibility, 7. visual polish.

The AI must avoid:

- overengineering,
 - unnecessary microservices early,
 - brittle assumptions about external APIs,
 - overly complex AI where rules are enough,
 - and designs that depend on perfect government infrastructure.
-

46. Final Instruction to the AI Builder

Build a system that behaves like a **serious civic operations platform**, not a hackathon toy.

The system should show the following qualities:

- it is fast,
- it is clear,
- it is accountable,
- it is resilient,

- it is mobile-ready,
 - it is easy for staff to use,
 - it is easy for citizens to file complaints,
 - and it is strong enough to be presented to government leadership.
-

47. Closing Statement

This blueprint is designed to be directly useful to an AI builder or engineering team. It defines the product vision, the functionality, the stack, the workflow, the security posture, the deployment strategy, the testing plan, the GitHub process, and the execution order.

If implemented faithfully, it should produce a solution that feels like a government-grade civic command center rather than a standard web app.